COMP/EECE 7/8740 Neural Networks

Topics:

- Alternative of Fully Connected (FC) layers
  - Why?/ What are alternatives?
- Transfer Learning (TL)
- Model Ensemble

Md Zahangir Alom
Department of Computer Science
University of Memphis, TN

Learning with Alternative of FC layers

- Pre-processing

- Initialization
- Advanceda activation function
- Regularization
- Batch normalization
- Loss functions
- Optimization techniques

- **Alternative FC**
- **Average pooling**
- **Global Average pooling**

Inputs

Related to the DNN models implementation and ensuring efficient training

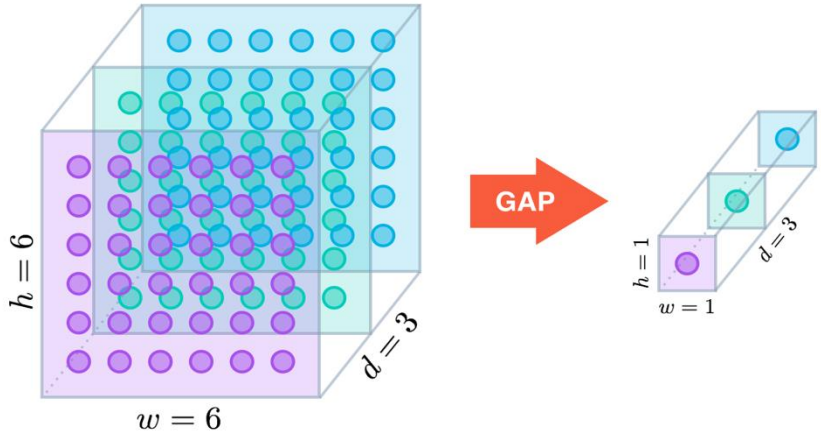
Related to classification & output layers

Why Alternative of Fully Connected (FC) layers?

- FC layers utilize the **maximum number of parameters**
- Alternative approaches help **to build optimum and efficient** deep learning architectures
- Why it's matter:
  - Ultimate goal is to achieve better accuracy with **minimum computational cost**
  - Speed up the training process, ensure **better accuracy, and help regularize** the models
  - Help improve **generalization ability**
- What are the **alternative methods?**
  - Global average pooling (GAP)
  - Go for (1x1) Convolution

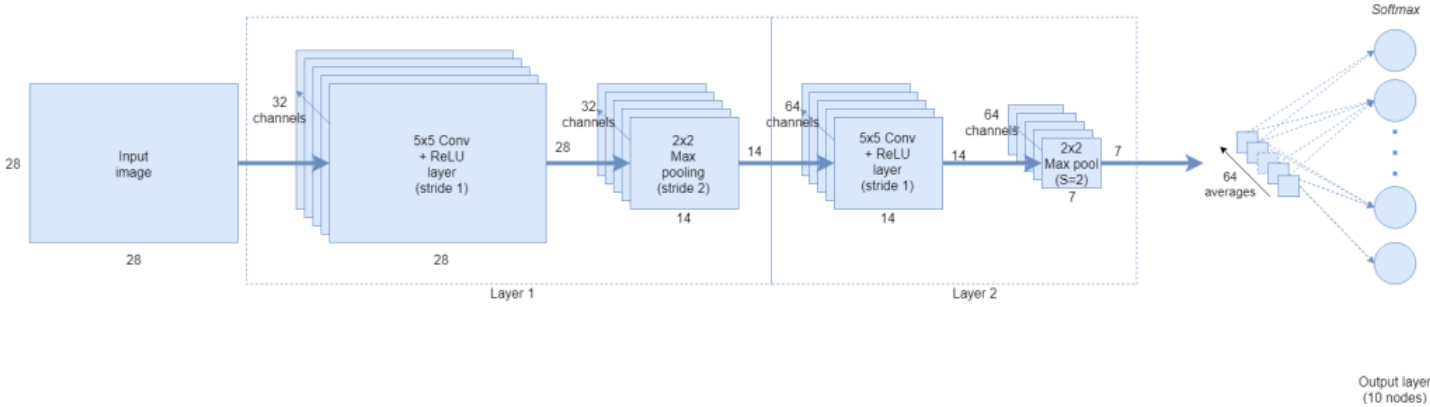
Global Average Pooling (GAP)

- Global average pooling calculates the **average output of each feature map in the previous layer**.
- Fairly simple and reduces the **data significantly and prepares the model for final classification**
  - Example: if you have outputs feature maps: 7x7x64, the GAP layer transform the dimension from **(7,7,64) to (1,1,64)** by performing operation on 7x7 pixels.



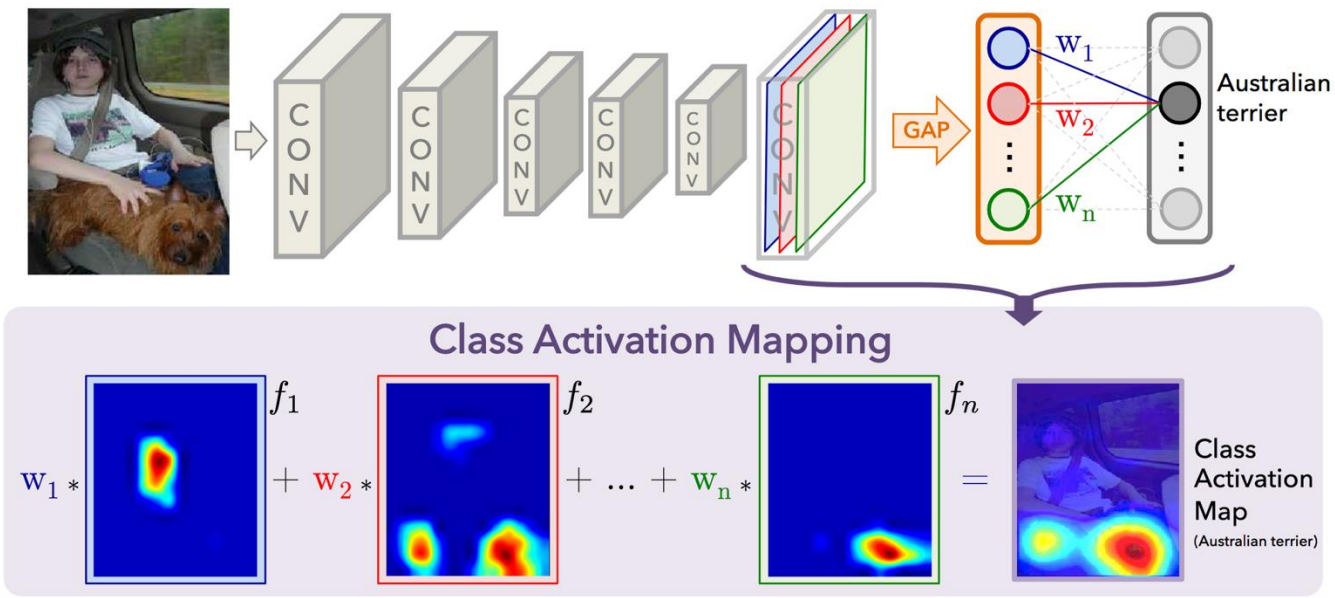
Advantages of GAP

- Advantages of GAP over fully connected layer:
  - **Removed large number of parameters** from the model
  - The elimination of all these trainable parameters also **reduces the tendency of overfitting** while fully connected layer requires the dropout
  - Removing fully connected classification layers **forces the feature maps to be more closely related to the classification categories**
  - Due to the averaging operation over the feature maps, this makes the model **more robust to spatial transformation** in the data.



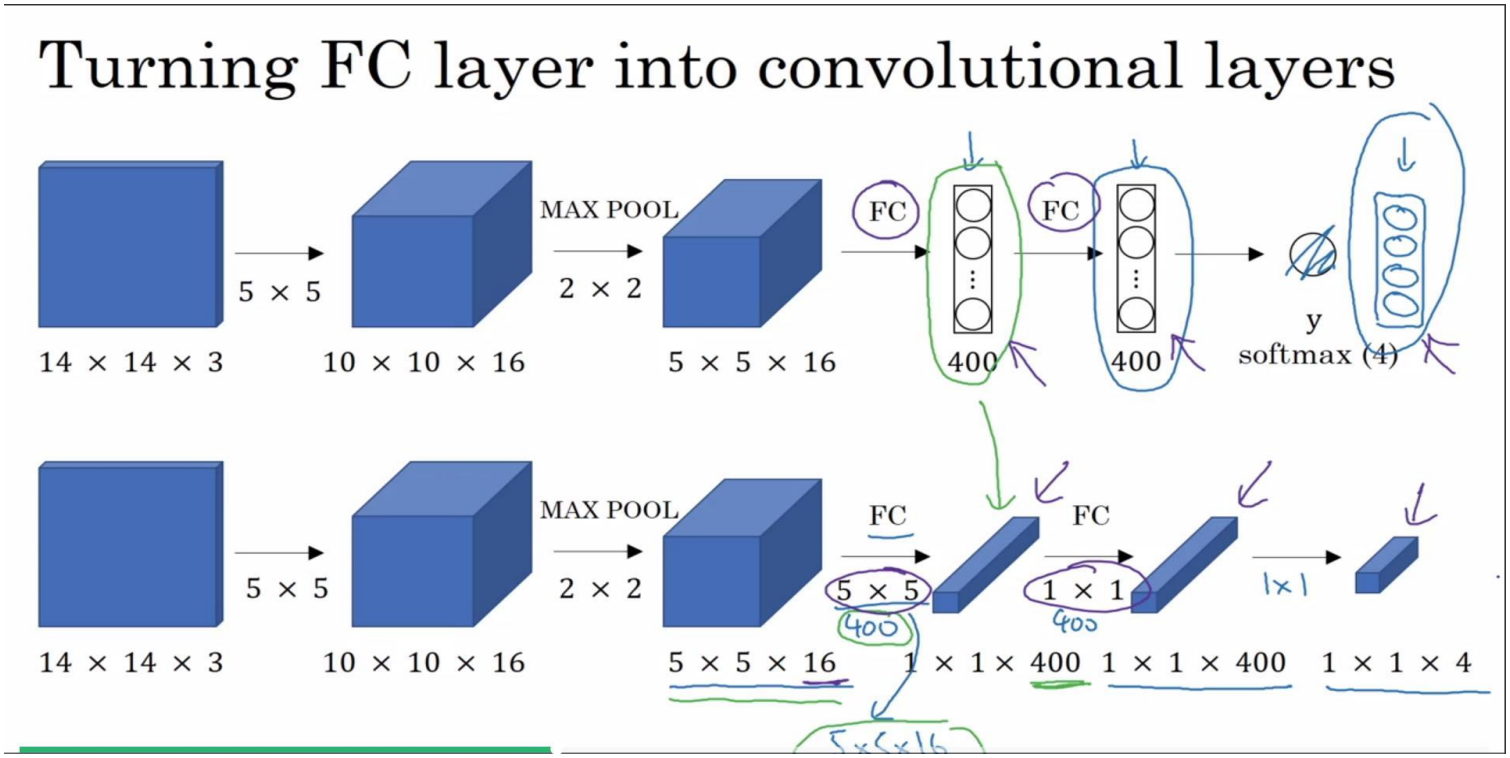
Where to apply GAP?

- Designing the models for :
  - Classification / Segmentation (regression)
  - **Very popular** for
    - Detection (classification + regression)
    - Localization and
    - Attention mechanism.



Go for (1x1) Convolution

- After having the feature maps before the fully connected layer, **you can apply (1x1) convolution with number of feature maps you want**

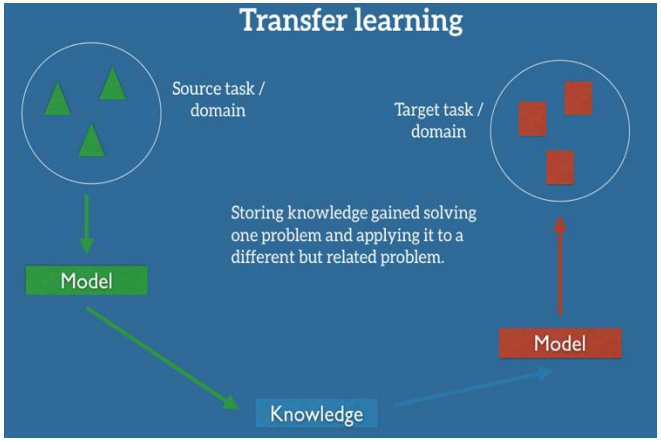


What is **Transfer Learning(TL)?**

Transfer learning is the idea of

- Overcoming the isolated **learning paradigm** and
- Utilizing knowledge **acquired for one task to solve related ones.**

- Some simple examples would be,
  - Know how to ride a motorbike → Learn how to ride a car
  - Know how to play classic piano → Learn how to play jazz piano
  - Know math and statistics → Learn machine/deep learning



The transfer learning setup

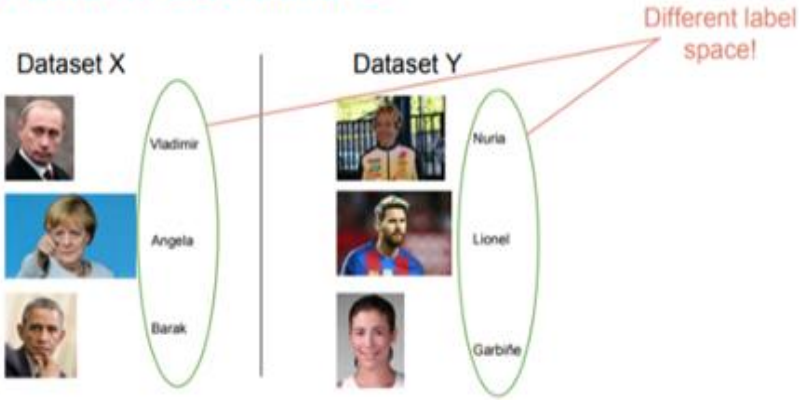
Image source : https://ruder.io/transfer-learning/

Domain and Task in Transfer Learning

- Clarity on the difference between the terms ***domain*** **and** ***task***

If two domains are different, they may have different feature spaces or different marginal distributions

If two tasks are different, they may have different label spaces or different conditional distributions



Pan, Sinno Jialin, and Qiang Yang. "A survey on transfer learning." *IEEE Transactions on knowledge and data engineering* 22, no. 10 (2009): 1345-1359.

Domain Adaptation using Transfer Learning (TL)

- **Traditional ML:**
  - **Train a model A** on this dataset and expect it to perform well on unseen data of the same task or domain
  - **Another domain or task B** : labeled again, and train the model
  - Traditional supervised learning care about training a reliable model, example : **pedestrian detection with day-time images**.

TL idea : instead of training deep network from scratch for your tasks:

- Take a network trained on a different domain or task
- Adapt it for your domain or task

Variation of TL:

- Same domain, different task
- Different domain, same task

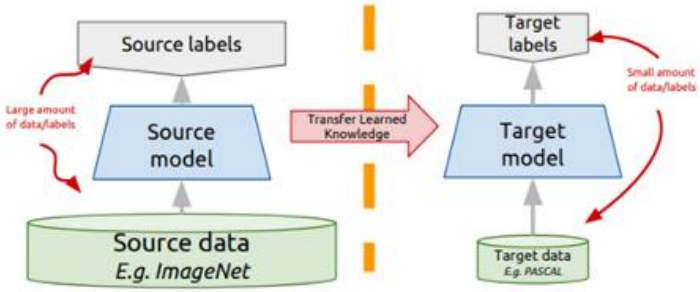
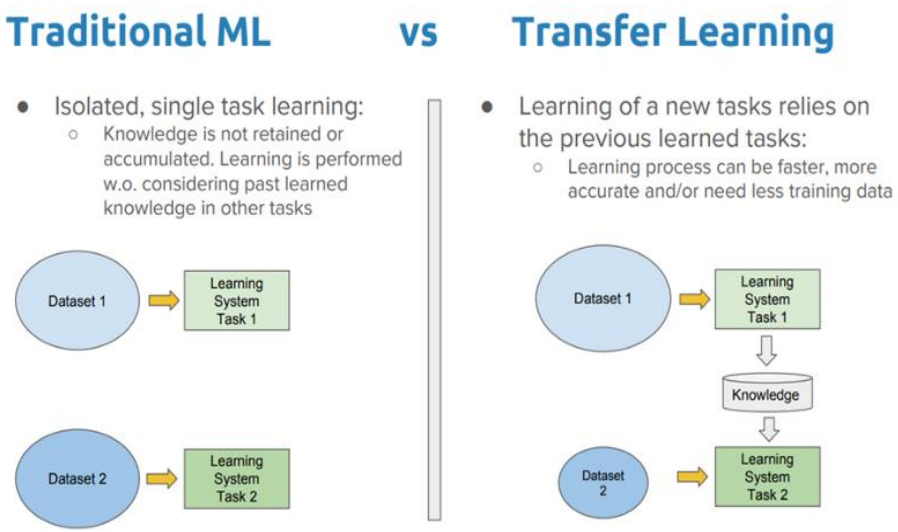


Image source : https://ruder.io/transfer-learning/

What to transfer in Transfer Learning (TL)?

- Traditional learning is isolated and occurs purely based on **specific tasks, datasets and training separate isolated.**
- **Portion of knowledge** is source-specific and what is common between **the source and the target.**
- Transfer learning **leverage knowledge (features, weights etc.)** from previously trained models for training newer models.



Traditional Learning vs Transfer Learning

Why Transfer Learning?

- **Myth:** you can't do deep learning unless you have a million of labeled example for a problem
- Most models solve **complex problems** need a whole lot of data, and **getting vast amounts of labeled data for supervised models can be really difficult.**

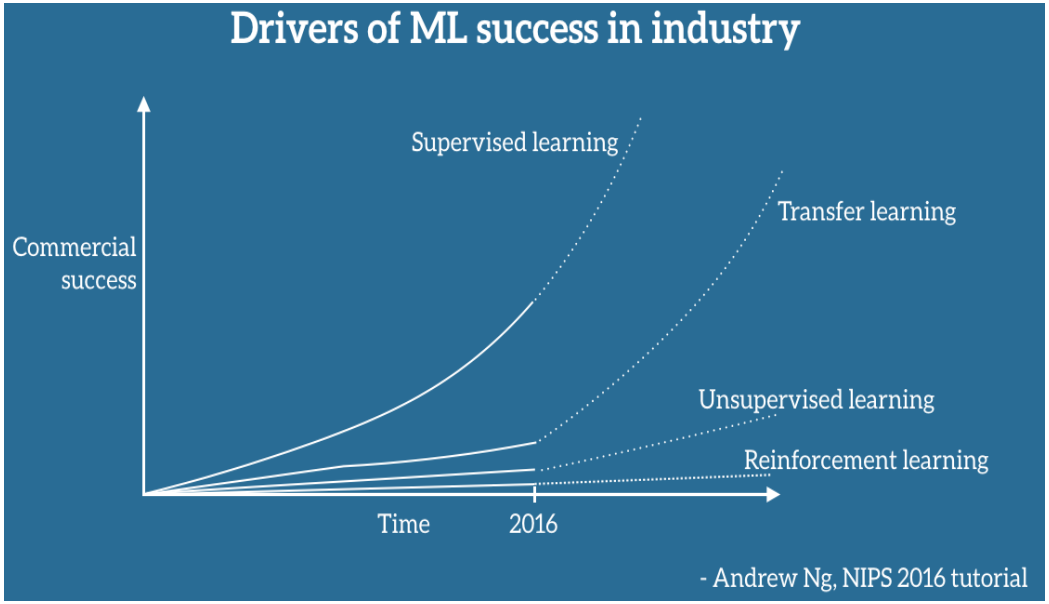
- **When you don't have enough computing resources + data**
  - **For examples :**
    - For NLP, GTP2/3 can be fine tune for your own tasks
    - For image generation, stable diffusion can be fine tuned with your datasets.

- TL can help the model to **generalize better than task specific model**
- **For task specific applications: incremental learning with new labeled examples,** help improving performance and generalization ability.

Image source : https://ruder.io/transfer-learning/

Why Transfer Learning?

- **True Artificial General Intelligence (AGI)**, transfer learning is something which data scientists and researchers believe can further our progress towards AGI
- In famous book, *Deep Learning*, **Goodfellow** et al refer to **transfer learning in the context of generalization.**
- Transfer learning will be -- after supervised learning -- the **next driver of ML commercial success** (Andrew Ng) in NIPS 2016



Drivers of ML industrial success according to Andrew Ng

Image source : https://ruder.io/transfer-learning/

When not to Transfer?

- **When not to Transfer:** transferring knowledge may **make matters worse than improving anything (also known as negative transfer).**

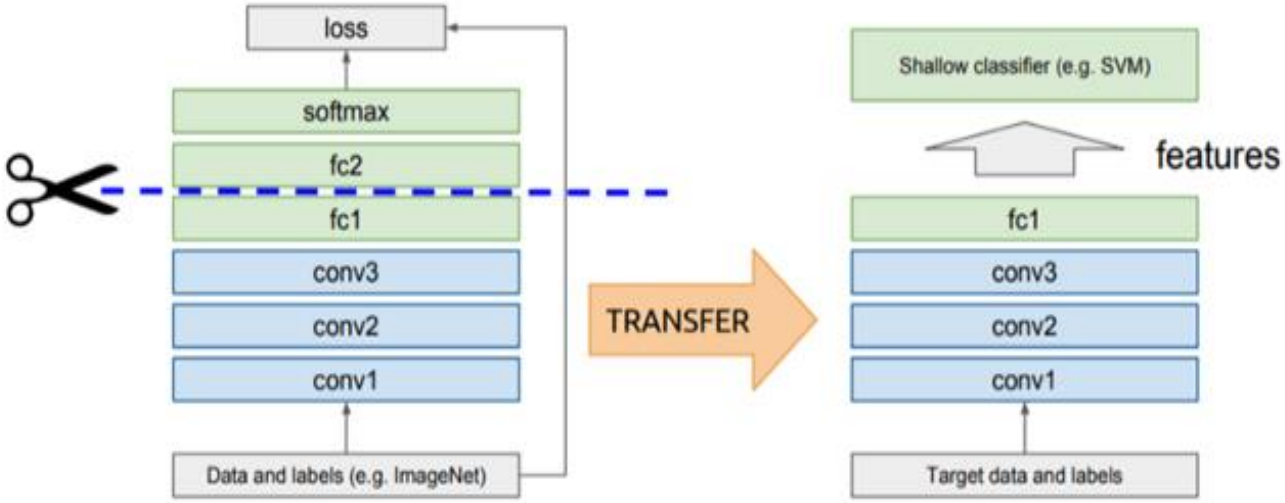
How to transfer : Deep Transfer Learning Strategies

- Identifying ways of actually transferring the **knowledge across domains/tasks.**
  - Off-the-shelf pre-trained models as feature extractors
  - Off-the-shelf features
  - Fine-tuned supervised domain adaptation
  - Freeze or fine-tune

Off-the-shelf Pre-trained Models as Feature Extractors

Idea: use outputs of one or more layers of a network trained on a different task as generic feature detectors. Train a new shallow model on these features.

Assumes that $D_S = D_T$



Transfer Learning with Pre-trained Deep Learning Models as Feature Extractors

Transfer learning with CNNs is pervasive…

(it's the norm, not an exception)

Object Detection
(Fast R-CNN)

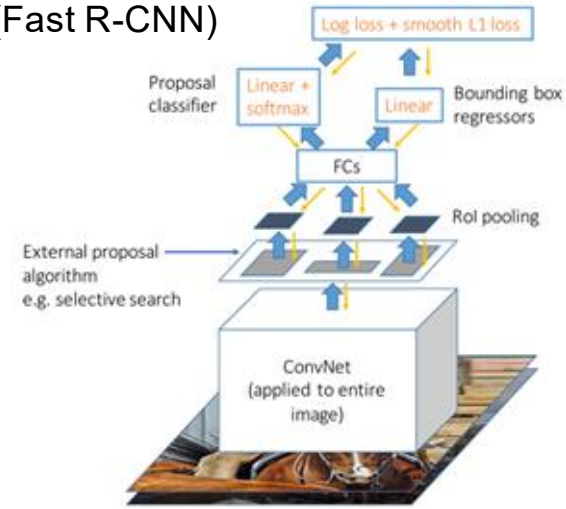
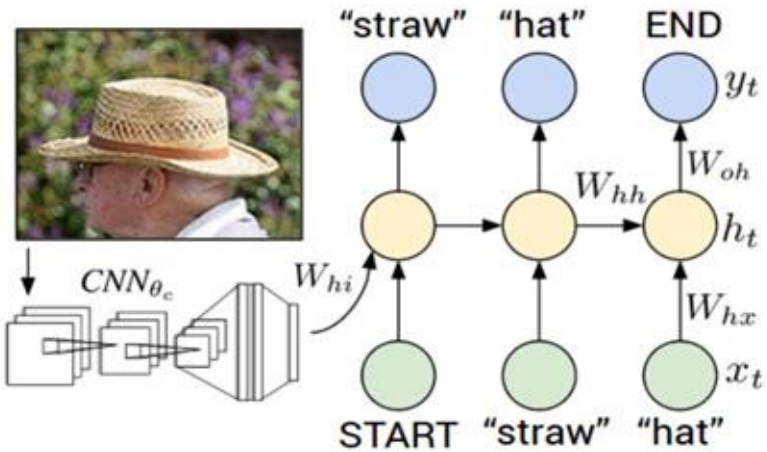


Image Captioning: CNN + RNN



Girshick, "Fast R-CNN", ICCV 2015

Figure copyright Ross Girshick, 2015. Reproduced with permission.

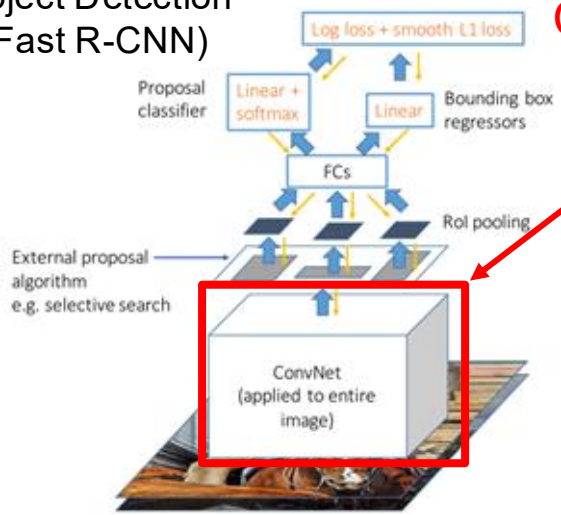
Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015

Figure copyright IEEE, 2015. Reproduced for educational purposes.

Transfer learning with CNNs is pervasive…

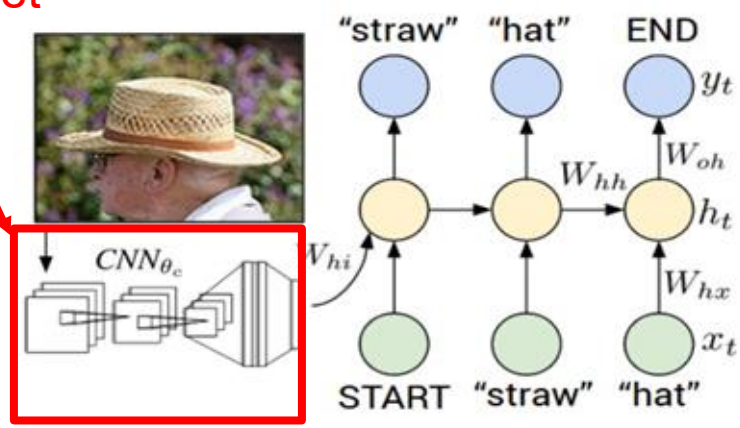
(it's the norm, not an exception)

Object Detection
(Fast R-CNN)



CNN pretrained on ImageNet

Image Captioning: CNN + RNN



Girshick, "Fast R-CNN", ICCV 2015

Figure copyright Ross Girshick, 2015. Reproduced with permission.

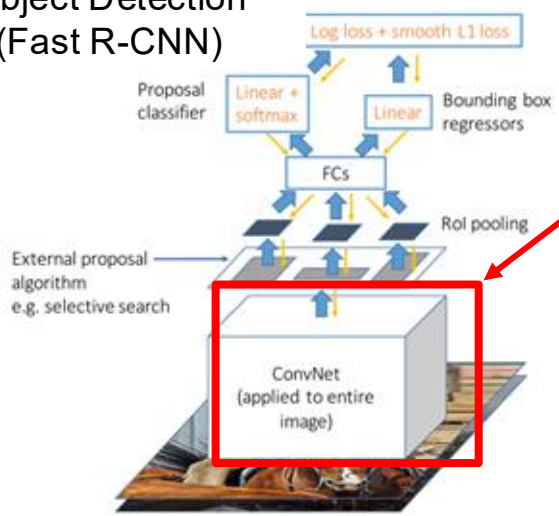
Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015

Figure copyright IEEE, 2015. Reproduced for educational purposes.

Transfer learning with CNNs is pervasive…

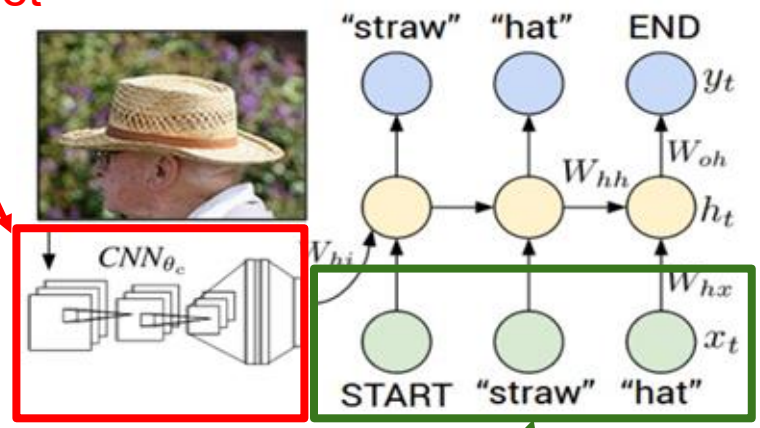
(it's the norm, not an exception)

Object Detection
(Fast R-CNN)



CNN pretrained on ImageNet

Image Captioning: CNN + RNN



Word vectors pretrained with word2vec

Girshick, "Fast R-CNN", ICCV 2015

Figure copyright Ross Girshick, 2015. Reproduced with permission.

Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015

Figure copyright IEEE, 2015. Reproduced for educational purposes.

Off-the-shelf features

Works surprisingly well in practice!

Surpassed or on par with state-of-the-art in several tasks in 2014

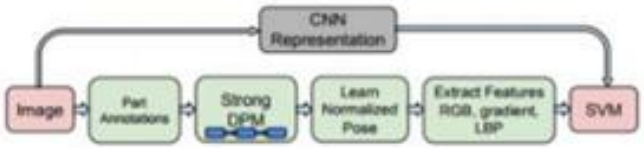
Image classification:

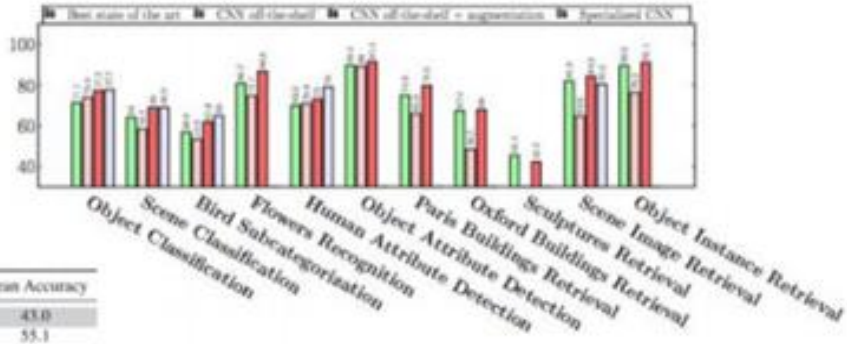
- PASCAL VOC 2007
- Oxford flowers
- CUB Bird dataset
- MIT indoors

Image retrieval:

- Paris 6k
- Holidays
- UKBench

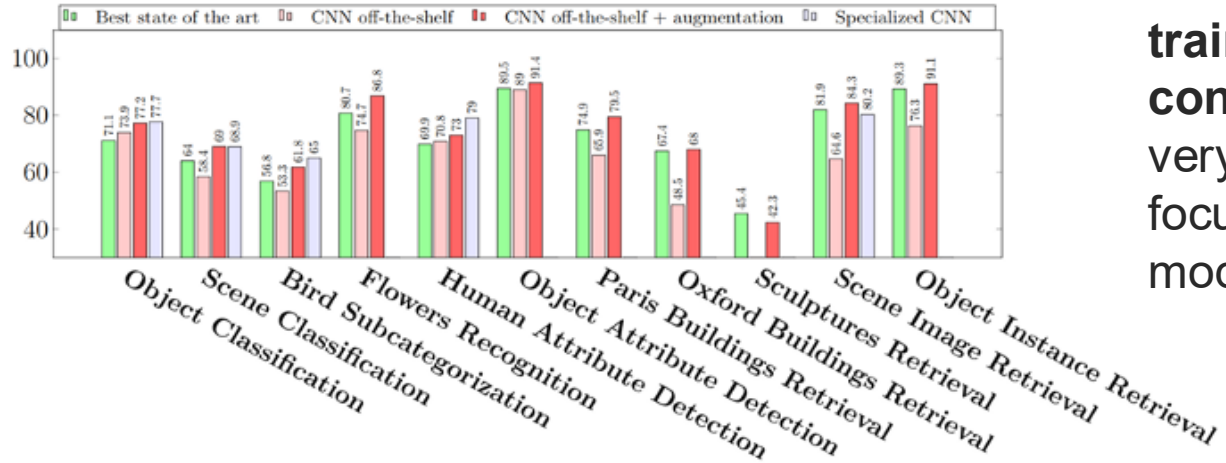
(Trained to perform object classification on ILSVRC13)





Method	mean Accuracy
HSV [27]	43.0
SIFT internal [27]	55.1
SIFT boundary [27]	32.0
HOG [27]	49.6
HSV+SIFTi+SIFTb+HOG(MKL) [27]	72.8
BOW(4000) [14]	65.5
SPM(4000) [19]	67.4
FLH(100) [14]	72.7
BiCos seg [7]	79.4
Dense HOG+Coding+Pooling[7] w/o seg	76.7
Seg+Dense HOG+Coding+Pooling[7]	80.7
CNN-SVM w/o seg	74.7
CNNaug-SVM w/o seg	**86.8**

Oxford 102 flowers dataset



Based on the red and pink bars in the figure, the **features from the pre-trained models consistently out-perform** very specialized task-focused deep learning models.

Fine-tuning: supervised domain adaptation

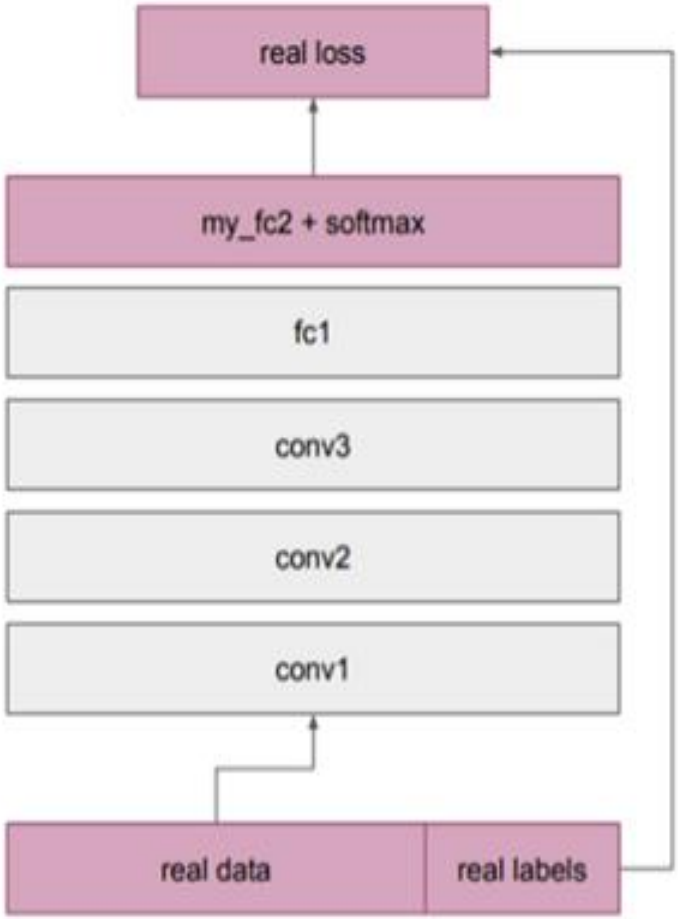
Train deep net on "nearby" task for which it is easy to get labels using standard backprop

- E.g. ImageNet classification
- Pseudo classes from augmented data
- Slow feature learning, ego-motion

Cut off top layer(s) of network and replace with supervised objective for target domain

Fine-tune network using backprop with labels for target domain until validation loss starts to increase

Aligns D_S with D_T



Freeze or fine-tune?

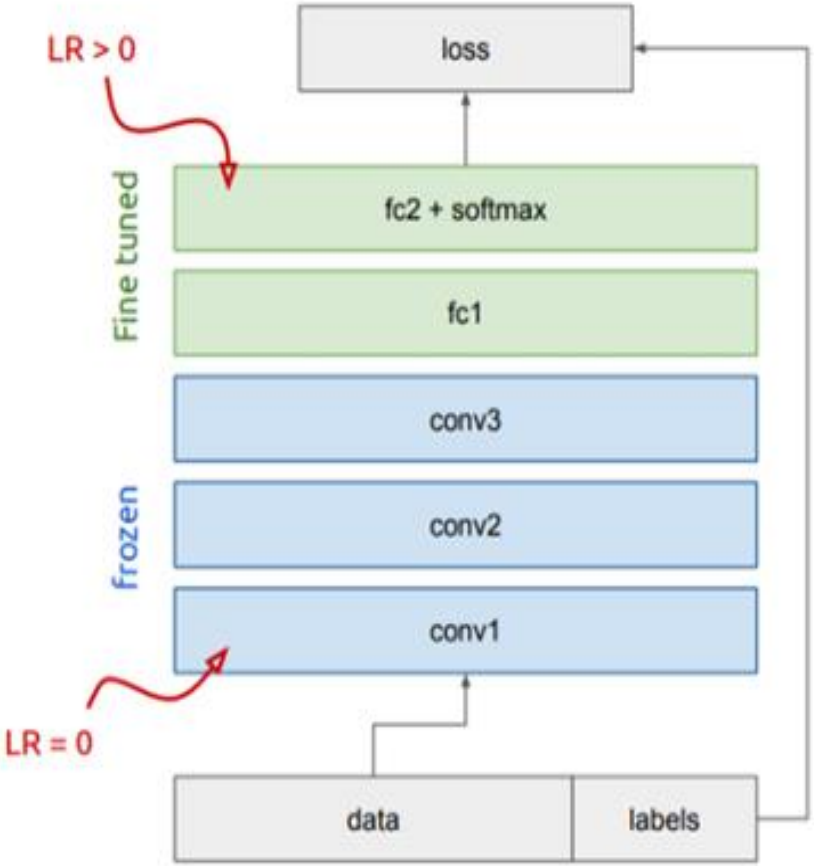
Bottom *n* layers can be frozen or fine tuned.

- **Frozen**: not updated during backprop
- **Fine-tuned**: updated during backprop

Which to do depends on target task:

- **Freeze**: target task labels are scarce, and we want to avoid overfitting
- **Fine-tune**: target task labels are more plentiful

In general, we can set learning rates to be different for each layer to find a tradeoff between freezing and fine tuning

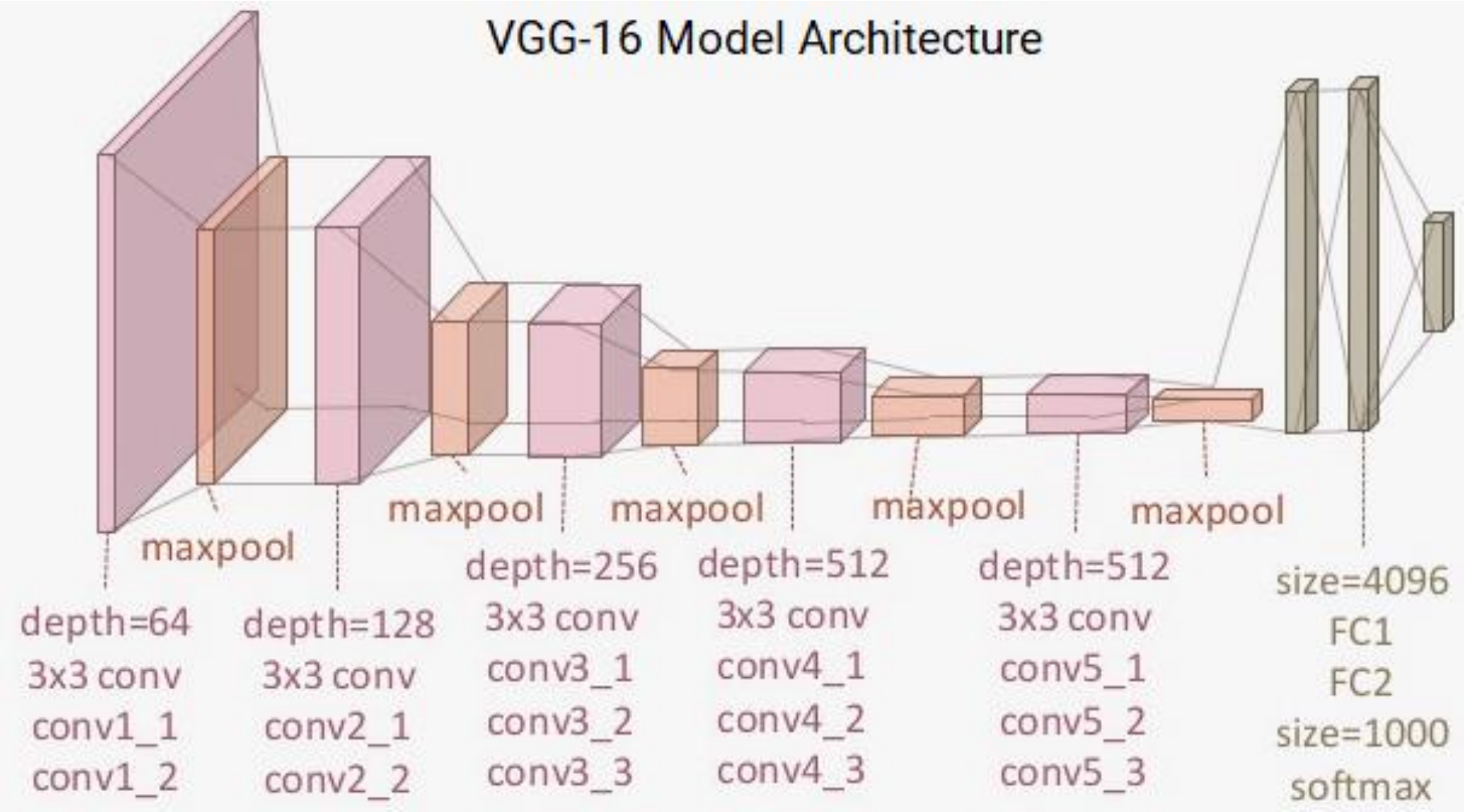


Where to find pre-trained models?

- One of the fundamental requirements for transfer learning is the presence of models that **perform well on source tasks**
- Many of the state-of-the art deep learning architectures have been **openly shared by their respective teams**
- ***For computer vision/NLP/ SaaS application (modelzoo/huggingface)***, you can leverage some popular models including,

  - VGG-16
  - VGG-19
  - Inception V3
  - XCeption
  - ResNet-50
  - Transformers and so on.

VGG-16 pretrained Model

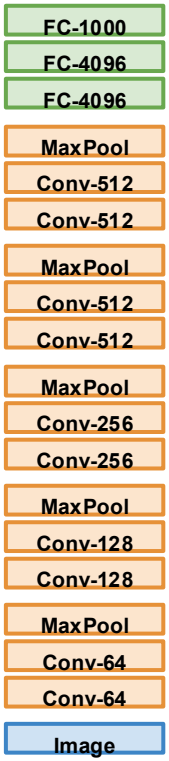


Pre-trained Models

- ***For natural language processing*** tasks, things become more difficult due to the varied nature of NLP tasks. You can leverage word embedding models including,
    - Word2Vec
    - GloVe
    - FastText
- Recently, there have been some excellent advancements towards transfer learning for NLP. Most notably,
    - Universal Sentence Encoder by Google
    - Bidirectional Encoder Representations from Transformers (BERT) by Google

Transfer Learning with CNNs

1. Train on Imagenet



Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014

Razavian et al, “CNN Features Off-the-Shelf: An Astounding Baseline for Recognition”, CVPR Workshops 2014

Transfer Learning with CNNs

1. Train on Imagenet

2. Small Dataset (C classes)

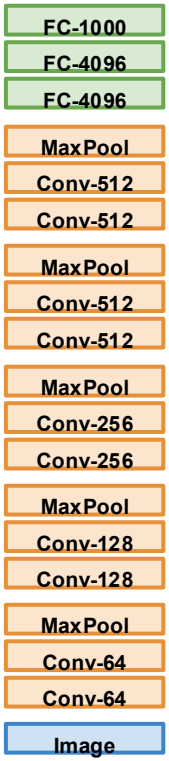
Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014

Razavian et al, “CNN Features Off-the-Shelf: An Astounding Baseline for Recognition”, CVPR Workshops 2014

Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n

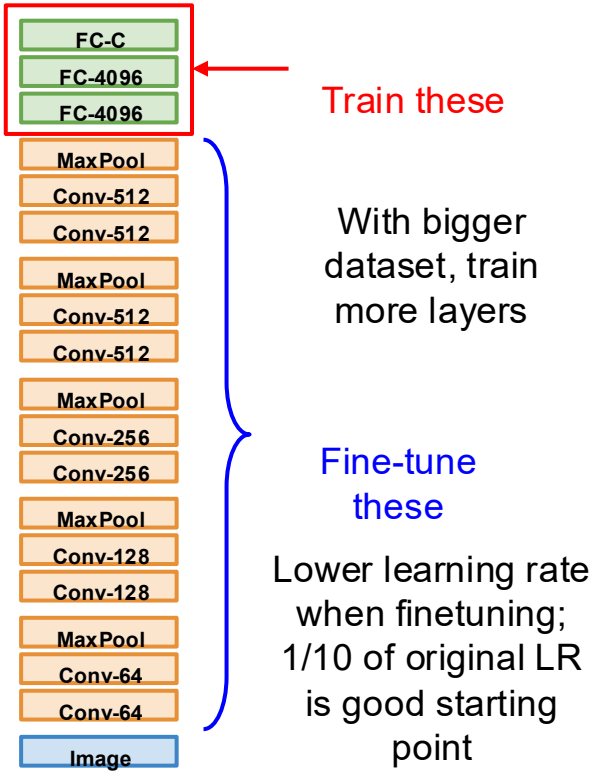
Transfer Learning with CNNs

1. Train on Imagenet



2. Small Dataset (C classes)

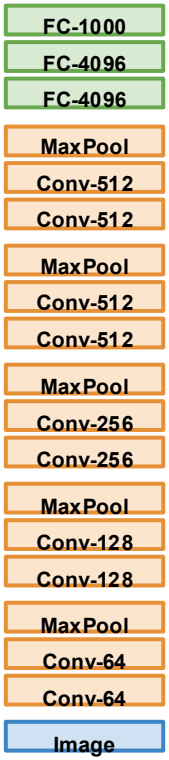
3. Bigger dataset



Donahue et al, “DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition”, ICML 2014

Razavian et al, “CNN Features Off-the-Shelf: An Astounding Baseline for Recognition”, CVPR Workshops 2014

Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n

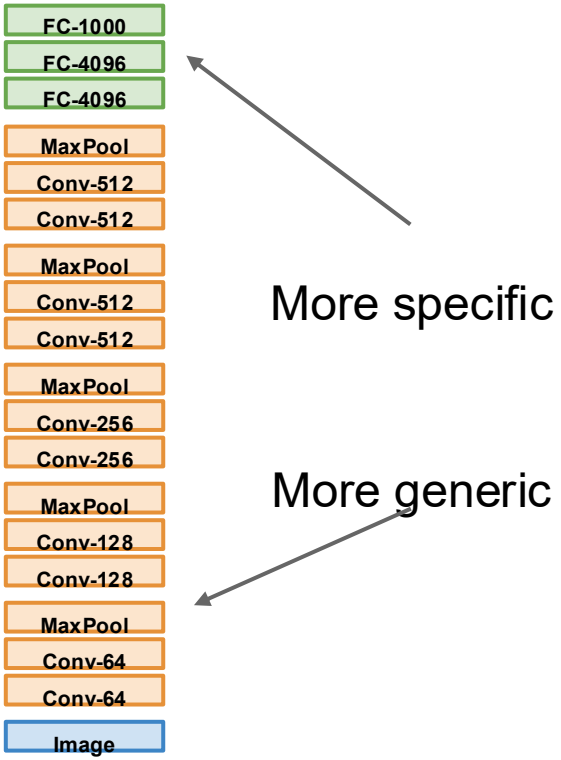


More specific

More generic

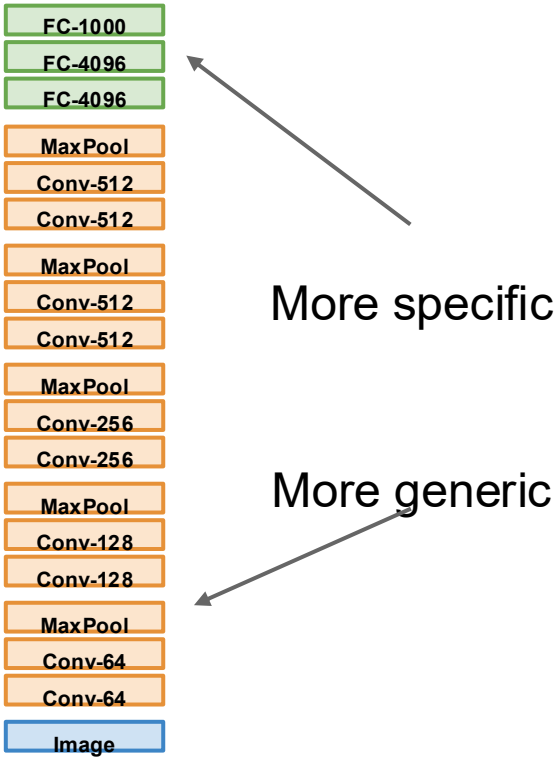
	very similar dataset	very different dataset
very little data	?	?
quite a lot of data	?	?

Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n



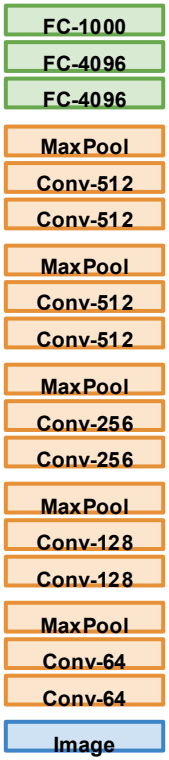
	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer	?
quite a lot of data	Finetune a few layers	?

Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n



	very similar dataset	**very different dataset**
very little data	Use Linear Classifier on top layer	You're in trouble… Try **linear classifier from different stages**
quite a lot of data	Finetune a few layers	Finetune a larger number of layers

Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n



More specific

More generic

	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer **(Freeze weights and train linear classifier** from top level features)	You're in trouble… Try linear classifier from different stages Fine-tune all layers **(faster convergence and better generalization)**
quite a lot of data	**Finetune a few layers** (Freeze weights and train linear classifier from non-top level features)	**Finetune a larger number of layers** (pretrain for **improved convergence speed)**

Takeaway for your projects and beyond:

Have some dataset of interest but it has < ~1M images?

1. Find a very large dataset that has similar data, train a big ConvNet there
2. Transfer learn to your dataset

"Model Zoo" a repository of pretrained models, so you don't need to train the models..

Keras : https://modelzoo.co/framework/keras

TensorFlow: https://github.com/tensorflow/models

PyTorch: https://github.com/pytorch/vision

Slide Credit: Fei-Fei Li, Justin Johnson, Serena Yeung, CS 231n

Ensemble Learning Methods for Deep Learning

- **What is an ensemble method?**
  Ensemble models in machine **learning combine the decisions from multiple models** to improve the overall performance.

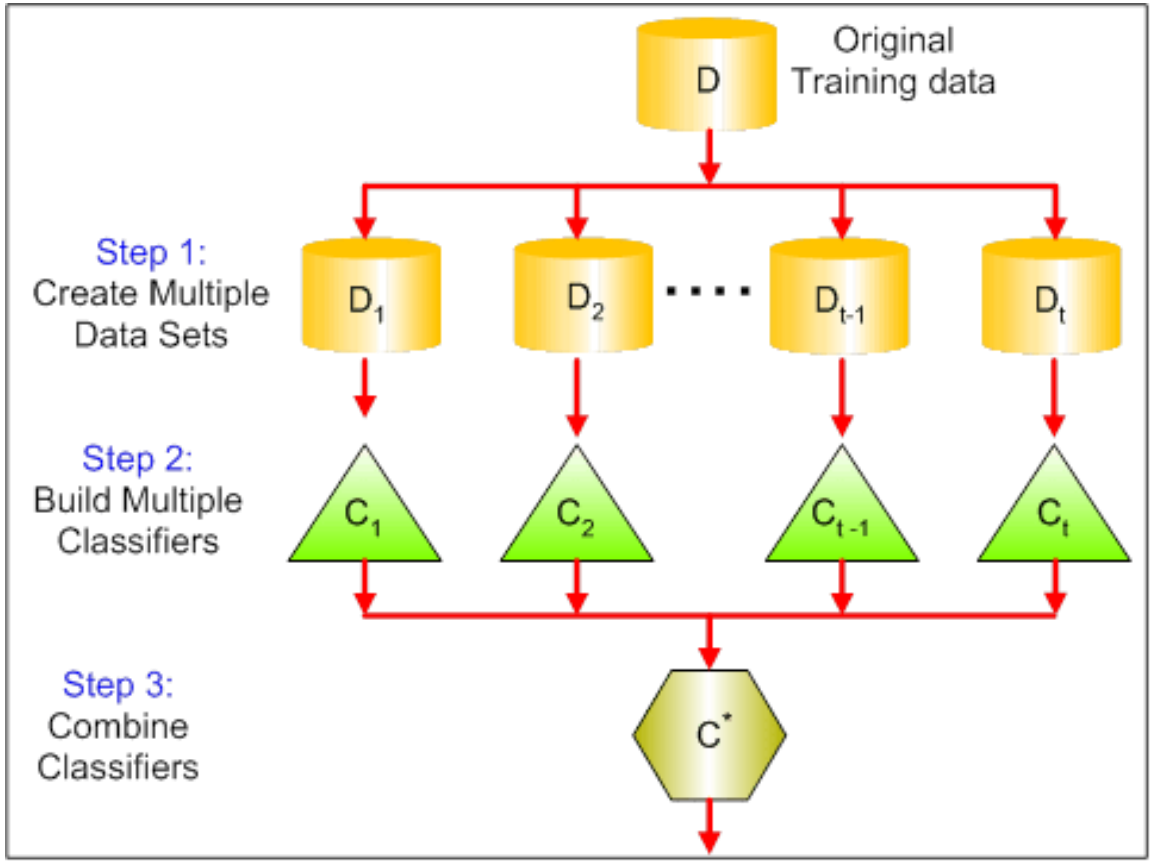
- **Benefits:**
  - The main causes of error in learning models are **due to noise, bias and variance.**
  - Ensemble methods **help to minimize these factors.**

- **Methods:** bagging, boosting and stacking

What is Bagging?

Bagging (Bootstrap Aggregating) is an ensemble method.

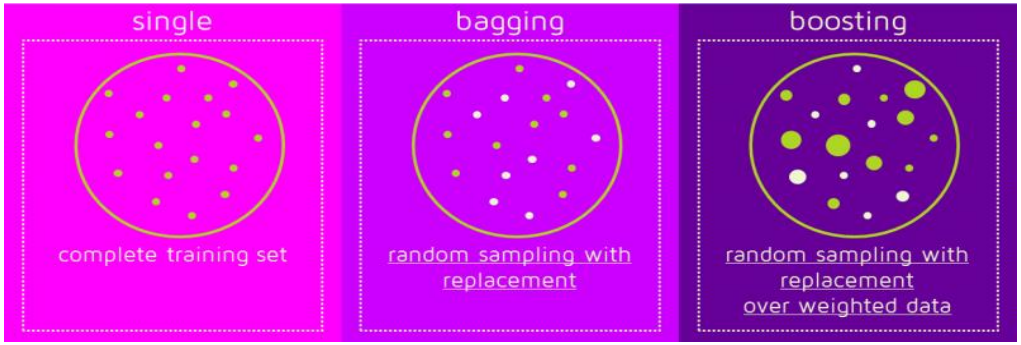
Boosting provides sequential learning of the predictors.



Bagging example

Bagging versus boosting

- Bagging, any element has the same probability to appear in a new data set.
- Boosting the observations are weighted and therefore some of them will take part in the new sets more often (help to reduce noise vs signal ratio)



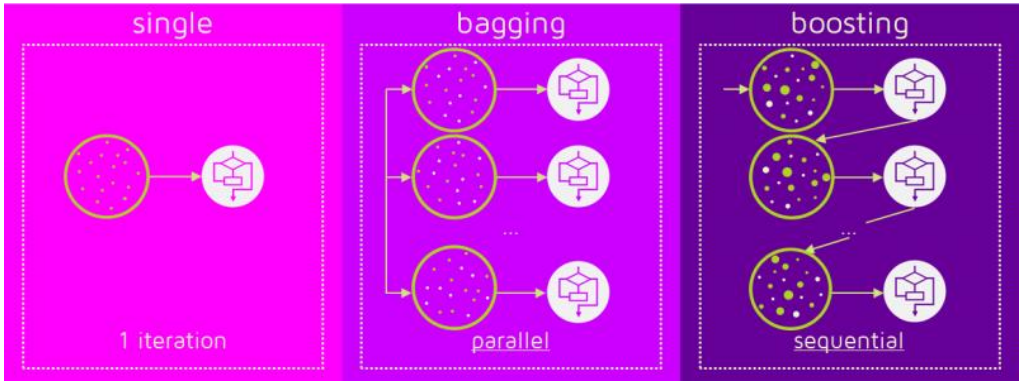
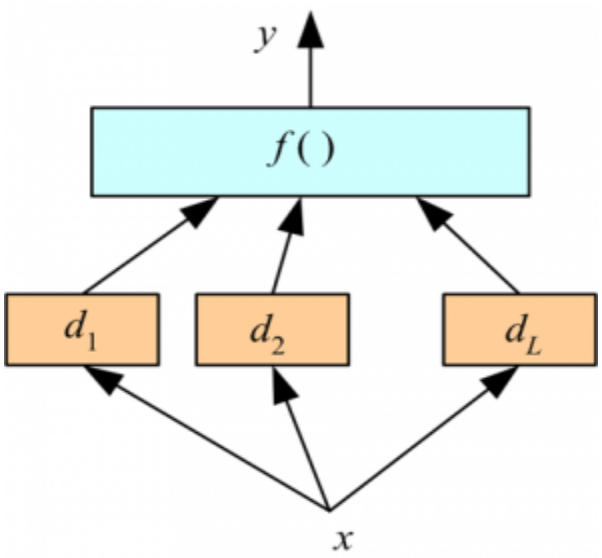


Image source: https://quantdare.com/what-is-the-difference-between-bagging-and-boosting/

What is stacking?

Stacking works in two phases.

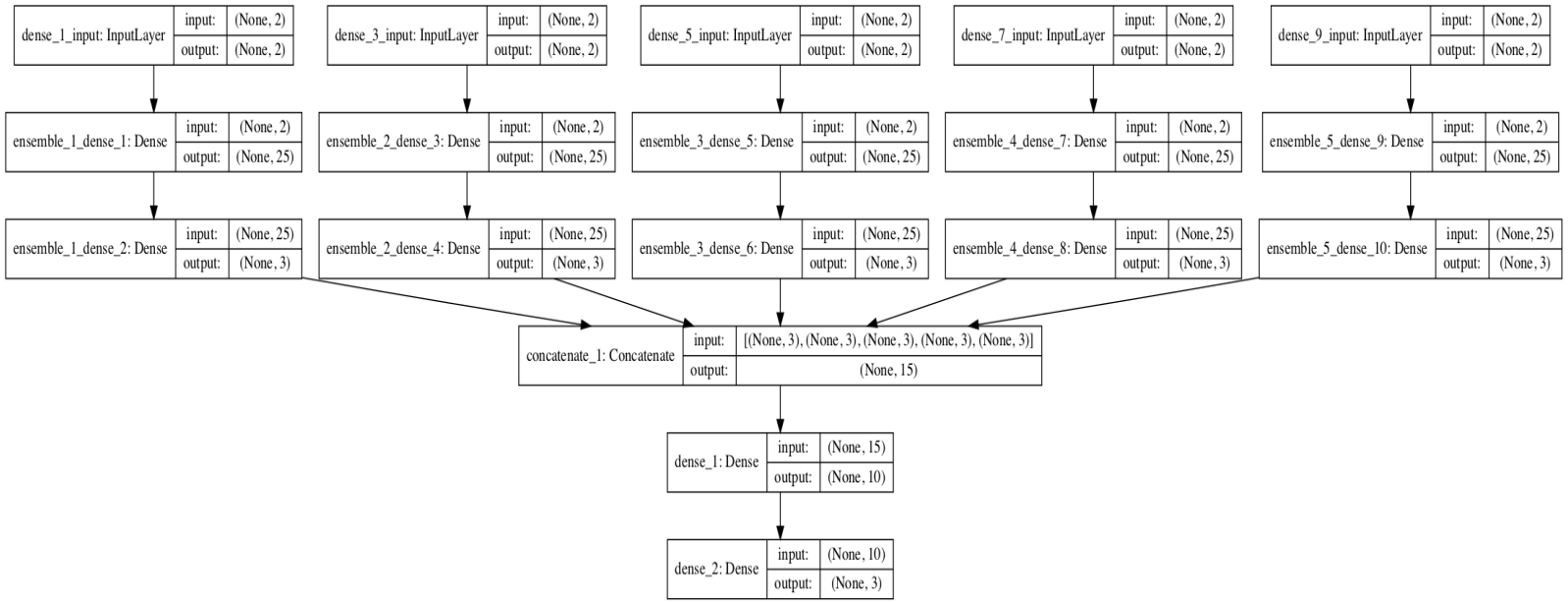
- First, we use multiple base classifiers to predict the class
- Second, **a new learner is used to** combine their predictions with the aim of reducing the generalization error



Stacking example

Ensemble Deep Learning Models

- Ensemble Methods :
  - Single Model Ensemble
  - Multi-Model Ensemble



Ensembled Model

How to find the best weights from ensemble model?

One of the most common challenge with ensemble modeling is to **find optimal weights to ensemble base models**.

- Assume equal **weight for all models and takes the average** of predictions

Some of the methods :

- Check the **cross-validation score (ratio of score) of identified base models to find the weight**.
- Find the algorithm to return the **optimal weight for base learners.**
  - Methods like:
    - Forward selection of learners
    - Selection with replacement
    - Bagging of ensemble methods

Advantages of Ensemble Models

- Using ensemble methods allows to produce **better predictions compared to a single model.**
- It is empirically evaluated that you can **get around 1~2% better performance over single model**
- **More stable and generalizable system**

Summary

- We have learned on the following the topic:
  - Alternative of fully connected layers
  - Transfer Learning (TL)
  - Model Ensemble

- **What's next?**
    - Segmentation (regression) models